

Day 4 — Computer Vision, MLOps, Active Learning and Feedback Loops

In-depth explanation of every concept, with analogies,
worked examples and delivery guidance

Written so that you can read it once and teach from it confidently across a full eight-hour day. Each concept has: what it means in plain terms, why it matters, the technical detail behind it, an analogy for the room, and the misunderstandings participants typically arrive with.

Enterprise AI Engineering Bootcamp · Companion to the 39-slide Day 4 presentation

How to Use This Document

Read end to end once before Day 4. Sections map to slide numbers in order. Four kinds of box appear throughout: **Analogy**, **How to say it**, **Watch for** and **Deeper detail**.

The claim Day 4 is built on

On industrial vision and ML problems, the largest available gains are almost never in the model. They are in the labels, the capture conditions, and the loop that feeds production experience back into training. Day 1 established the diagnostic method; today applies it to the two hardest industrial problem types and builds the operational machinery that keeps improvement running.

Timing guide

Slot	Content	Pacing notes
09:00 – 10:45	Module 1 — computer vision	Section 1.3 (size versus quality) is the one to protect. It carries the whole day's argument
10:45 – 11:30	Lab 1 — why does the model fail?	Step 4, the label audit, is what changes minds
11:30 – 13:00	Module 2 — CV error analysis	The highest-value hour of the day. Do not compress
13:45 – 14:45	Lab 2 — error analysis report	Push teams to lead with the label audit finding
14:45 – 16:15	Module 3 — MLOps	Frame as five questions, not a toolchain. Keep the tooling discussion out of it
16:15 – 17:00	Lab 3 — pipeline design	The rollback-time question exposes whether the design is real
17:00 – 17:45	Module 4 — active learning	The calibration dependency in 4.2 is easily missed and undermines everything
17:45 – 18:25	Lab 4 and assignment brief	Insist on the volume arithmetic

Module 1 — Computer Vision for Industrial Use Cases

1.1 The task types

Task	Output	Use when	Labelling cost
Image classification	One label per image	The whole image is pass or fail	Lowest
Object detection	Boxes with class and confidence	You need to know what and where	Moderate
Segmentation	Per-pixel class	Extent or area matters — corrosion coverage, weld length	Highest
OCR	Text from an image	Serial numbers, nameplates, displays	Low if templates are stable
Gauge reading	A numeric value	Analogue instruments without digital output	Moderate
Anomaly detection	Normal or anomalous, with a score	Defects are rare, varied, cannot be enumerated	Lowest — normal images only

The choice that saves the most effort. "If your defects are rare and varied, anomaly detection needs only images of good parts. That is a fundamentally easier data problem than collecting and labelling enough examples of every defect type — and for many inspection tasks it is both the honest answer and the better one." This is the same argument as Day 2's predictive maintenance case: when positives are rare, reformulate rather than fight the imbalance.

1.1b OCR and gauge reading

Two tasks industrial teams consistently underestimate, because they look solved and are not — on your images.

- **OCR on industrial surfaces.** General OCR is trained on documents. Your text is stamped, etched, embossed or laser-marked on metal, at an angle, under directional lighting, sometimes worn. What helps: control the lighting angle, restrict the character set, validate against a known serial format, and reject rather than guess at low confidence.
- **Gauge and display reading.** Fix the camera so the geometry is constant, calibrate the dial range once, and detect the needle rather than reading the whole image.

Watch for — parallax. A camera not perpendicular to a dial reads consistently high or low. Nothing in the model reveals it, and the error is systematic rather than random, so averaging does not help. It is fixed with a mounting bracket, not a network. A good example of a class of error where the honest answer is mechanical.

1.2 Model selection

What drives the choice: inference location and latency, defect size relative to image size, available labelled data, explainability requirement, and licensing. What actually determines performance: label consistency, capture control, resolution at the defect scale, and coverage of real variation.

How to say it. "A well-controlled capture setup with a modest model beats an excellent model on inconsistent images, reliably. Benchmark leaderboard position is measured on datasets that look nothing like yours."

Raise licensing early. Several widely-used vision frameworks and pretrained weights carry copyleft licences whose terms can extend to network-delivered services, which may require a commercial licence for internal enterprise deployment. Others

restrict pretrained weights to non-commercial research use. Establish the licence position for every model and dataset before it enters a production pipeline. Raise this as a required check rather than naming any product as a problem — it earns credibility with their legal and procurement functions and it is genuinely important.

1.3 Dataset size versus dataset quality

The most consequential table of Module 1, and the one that connects directly to Day 1.

Situation	More data	Better labels	Better capture
Labels are inconsistent	No effect — adds inconsistency	Large effect	No effect
Defect smaller than the model can resolve	No effect	No effect	Large effect
A defect class has 12 examples	Large — targeted collection	Moderate	No effect
Lighting varies uncontrolled between shifts	Small	No effect	Large effect
Model overfits a small clean set	Moderate	Small	Small
Two inspectors disagree on borderline parts	No effect	Large effect	No effect

The point to make. "Four of these six rows are not solved by more data. Before you fund a collection programme, find out which row you are in. That is Day 1's error analysis, applied to vision."

1.4 Labelling guidelines — where the ceiling is set

1. **Measure agreement first.** Two experienced inspectors label the same 100 images independently. Report raw agreement and Cohen's kappa. This is your ceiling.
2. **Write the definition down.** What qualifies, what does not, what to do at the boundary.
3. **Include visual examples of every borderline case.** A picture of the smallest mark that counts as a defect is worth pages of definition.
4. **Define an ambiguous category.** Give annotators somewhere to put genuinely unclear cases rather than forcing a guess that becomes training noise.
5. **Re-label a subset and measure again.** If agreement did not rise, iterate before labelling at scale.
6. **Version the guideline.** Labels from different guideline versions are not comparable.

Analogy — the inspection standard. No quality department would run a manual inspection process without a written standard and boundary samples. Every serious manufacturing operation has a limit sample cabinet. A labelling guideline is exactly that, for the training data — and teams that would never accept undefined manual inspection routinely accept undefined labelling.

How to say it. "Your model's realistic ceiling is roughly where your inspectors agree with each other. If two inspectors agree 78% of the time, a 95% target is not ambitious — it is asking the model to be more consistent than the ground truth it learns from."

1.5 Augmentation

Legitimate: transformations representing variation that genuinely occurs — brightness and contrast within the real range, small rotations if placement varies, noise and blur matching real sensor behaviour, scale if working distance varies.

Illegitimate: 90-degree rotation with a fixed camera and fixture, horizontal flip on an asymmetric part with fixed orientation, colour shifts on a monochrome setup, elastic distortion on a rigid metal component.

The framing that stops copy-the-tutorial-defaults. "Augmentation teaches the model what to ignore. Every augmentation is a claim about your process. Teaching it to ignore orientation when a mark on the left means something different from a mark on the right actively destroys information it needs. Check the claim with someone who knows the line before applying it to a million images."

The practical test: could this image plausibly have come out of your camera?

1.6 Production variation — four axes

Axis	What changes	Why it matters
Lighting	Shift changes, ambient light, ageing lamps	The most common cause of degradation after deployment
Camera and optics	Focus drift, dirt, replacement units, mounting shift	A replaced camera is a distribution shift nobody logs
Part presentation	Fixture wear, orientation tolerance, oil film	Small drifts accumulate into a new distribution
Product mix	New variants, revised parts, alternative suppliers	Invisible to monitoring until failures appear

The cheap recommendation to give them. "Every one of these is a change to the physical process that nobody records as a change to the AI system — and each will show as unexplained model degradation weeks later. Keep a change log linking physical process changes to model monitoring. It costs nothing and it is almost never in place."

Module 2 — CV Error Analysis and Improvement

2.1 The vision failure taxonomy

Failure mode	Signature	First hypothesis
Small object	Recall drops sharply with defect size	Increase resolution or tile before inference
Low contrast	Failures cluster on particular finishes or lighting	Improve capture lighting; add matched training data
Occlusion	Defect partly hidden	Add occluded examples; consider segmentation
Class confusion	Two classes systematically swapped	The class definitions overlap — a labelling problem
Poor labelling	Annotators disagree on the failures	Rewrite the guideline with boundary examples, re-label
Class imbalance	One class few examples, poor recall	Targeted collection, class weighting, or merge
Domain shift	Performance fell after a physical change	Recapture training data under current conditions
Production drift	Gradual decline, no single cause	Scheduled evaluation on a fresh labelled sample
Threshold misplacement	The precision-recall mix is wrong	Cost-optimal threshold — no retraining needed

The line to emphasise. "Two of these — poor labelling and threshold misplacement — need no new data and no retraining. They are also the two teams check last."

2.2 Small objects

A defect occupying 20 pixels in a 4,000-pixel-wide image is 0.5% of the width. Most detection architectures downsample it out of existence before the detection head sees it.

Approach	How it works	Cost
Higher input resolution	Feed the network a larger image	Memory and latency rise sharply
Tiling	Overlapping tiles, inference on each, merged	Linear increase in inference count
Higher optical magnification	Change the physical setup so the defect is larger	Best result where feasible
Multi-scale architecture	Feature pyramid designed for scale variation	Moderate; a model choice
Two-stage inspection	Fast pass locates regions; slow pass inspects	Often the best latency-accuracy trade

The first question to ask. "How many pixels does the smallest defect you must catch occupy? Establish that before choosing anything. If the answer is under about twenty, the honest fix is optical rather than algorithmic."

2.2b Occlusion, contrast and the limits of the image

Condition	Why the model fails	What actually fixes it
Partial occlusion	The distinguishing feature is hidden	Change the fixture or add a viewpoint
Low contrast	Defect and surface differ by few grey levels	Directional or raking lighting, polarisers — an optical fix
Specular reflection	Highlights saturate the sensor	Polarisers, diffuse lighting, multiple exposures

Motion blur	The part moves during exposure	Shorter exposure with more light, or a strobe
Depth of field	Part of the surface is out of focus	Smaller aperture with more light, or telecentric optics

The diagnostic that costs an hour and settles the question. "Can a trained inspector reliably see the defect in this image? If not, the model cannot either, and no architecture or data volume will change that. Print twenty failure images and ask an inspector."

2.3 Class confusion

The confusion matrix shows two classes systematically swapped. The instinct is to add training data for both. What is usually happening is that the two class definitions overlap and your annotators were also swapping them — the model has learned the inconsistency faithfully.

The diagnostic: take 50 images the model confused, have two inspectors label them independently without seeing the model's answer. If the inspectors also disagree, the problem is the definition.

The move nobody considers — merging classes. If a distinction cannot be made reliably by trained humans, carrying it forward guarantees a ceiling. Merging feels like a retreat and is usually the honest simplification. Ask what decision actually depends on telling the two classes apart; frequently the answer is none, and the distinction was inherited from a taxonomy written for a different purpose.

2.4 From CV failures to testable hypotheses

Observed failure	Measurable hypothesis	Test
Recall poor on defects under 30 px	Tiling at 2× overlap raises small-defect recall from 0.52 to ≥ 0.70	McNemar on the small-defect subset
Classes C and D confused	Re-labelling under a revised guideline raises per-class recall by ≥ 0.06 for both	Paired comparison, fixed held-out set
Failures cluster on night shift	500 night-shift images bring night recall within 0.03 of day	Two-proportion test between slices
False rejects too costly	Threshold at 0.38 reduces total error cost by $\geq 30\%$	Cost curve on held-out data

Note that the fourth requires no new data, no labels and no training run. On many industrial systems it is the largest single available gain.

2.4b Threshold tuning and statistical comparison

1. **Establish both error costs.** Cost of a scrapped good part and cost of an escaped defect. Finance owns these numbers.
2. **Sweep the threshold** on held-out data and plot total cost.
3. **Check per class.** The cost-optimal threshold frequently differs by defect class.
4. **Confirm on a later time window.** A threshold tuned on old data may not hold.
5. **Compare models with McNemar.** Both models on the same images, each right or wrong — exactly McNemar's case.
6. **Report per slice.** An improvement that helps common classes and harms a rare critical one is not an improvement.

An idea most teams have not considered. "Per-class thresholds. A safety-critical defect class set to favour recall heavily, a cosmetic class set to favour precision — one model, two operating points, no retraining."

2.5 Human review for vision systems

Day 1's three-band design applied to inspection. Six design questions: what lands in review; who reviews; how fast it must be; what the reviewer sees; what happens to the decision; and what is measured.

The constraint that is far tighter than for documents. On a line running at two parts per second, a 12% deferral rate is 8,600 reviews per shift. Review must fit the cycle time or the part must be diverted. Do the arithmetic before choosing thresholds — a review design that exceeds capacity fails on day one.

Module 3 — MLOps for Production AI Systems

3.1 What MLOps is actually for

Not tooling for its own sake. Five specific questions it makes answerable: can you reproduce a result; can you roll back; can you tell it degraded; can you ship a change safely; can you retrain reliably.

How to open the module. "If the honest answer to any of these five is no, that is your MLOps priority. Do not start with the tooling — start with the unanswered question. Buying a platform to solve a discipline problem is how organisations end up with an expensive platform and the same problem."

3.2 Versioning — four things, bound together

What	Why	Consequence of omitting
Dataset version	The exact data used for a run	Results cannot be reproduced or compared
Feature version	The code that computed the features	Training-serving skew; silent invalidation
Model version	Weights, architecture, hyperparameters	Cannot roll back; cannot say what is serving
Label guideline version	The definition labels were created under	Labels from different eras silently incomparable

Deeper detail — why the binding matters. Rolling back a model without rolling back the feature code gives you a model running on inputs it was never trained for, which frequently performs worse than either version alone. Teams discover this during an incident, which is the worst possible time. Each model version must record the dataset, feature and guideline versions that produced it.

3.3 Experiment tracking

Record per run: dataset and feature versions, code commit, hyperparameters, all metrics including per-class and per-slice, duration and cost, who ran it, and **the hypothesis it was testing**.

The line that gets skipped and matters most. "A tracked run without a stated hypothesis is a record that something was tried. A tracked run with one is a record of what was learned — including the negative results, which are the ones teams repeat because nobody wrote down that they did not work."

3.4 CI/CD for machine learning

Seven stages: commit, validate, train, evaluate, gate, register, deploy.

What makes it MLOps rather than CI. "The gate. The pass criterion is set before the run, exactly as on Day 1: no slice may regress by more than a stated margin, the primary metric must exceed the current model, and every regression case must pass. A model that fails the gate does not get deployed and does not get argued into production. That is what stops quality drifting downward one small exception at a time."

3.4b Registry, serving and inference mode

- **Model registry.** Version and lineage, metrics per class and per slice, approval status and approver, which environment it serves, known limitations. The prerequisite for rollback.

- **Model serving.** Batching, warm instances, hardware match, concurrency limits, graceful degradation. For edge, add fleet update management.
- **Batch or real-time.** Inspection on a line is real-time by necessity. Maintenance scoring is frequently batch by choice — and teams build real-time anyway.

A concrete practice worth recommending. "Once a quarter, rebuild an older model from its recorded lineage and compare. Teams that assume reproducibility without testing it usually discover, during an incident, that an undocumented step existed in someone's notebook."

3.5 Deployment strategies

Strategy	How it works	Detects
Shadow	New model runs live; output not used	Real behaviour, latency, cost, disagreement rate — at zero user risk
Canary	Small traffic share to the new model	Real outcomes on a limited population
Blue-green	Two environments, traffic switched	Nothing new — it is a fast rollback mechanism
Staged rollout	Progressive increase with monitoring	Problems emerging only at scale or on specific segments
Champion-challenger	Challenger runs continuously alongside	Sustained comparison rather than a one-off test

For industrial vision, shadow deployment is unusually valuable: you can compare the new model's decisions against the current one *and* against inspector judgement on the same physical parts, before anything changes on the line.

3.6 Monitoring and retraining triggers

Signal	Indicates	Without labels?
Input distribution shift	Capture conditions or product mix changed	Yes — the earliest signal
Confidence distribution shift	The model is less certain	Yes
Deferral rate rising	More cases in the uncertain band	Yes
Reviewer override rate rising	Humans disagreeing more often	Yes, if captured
Accuracy on a fresh sample	Actual performance	No — requires labelling
Business outcome change	Escapes, scrap rate, warranty claims	Lagging, but the one that matters

Retraining triggers, set in advance: accuracy below the error budget on the fresh sample; a defined volume of new labelled data; a logged physical process change; or a scheduled cadence as a floor. Triggering on a single alarming week is how you retrain on noise.

Module 4 — Active Learning and Feedback Loops

4.1 Why active learning matters industrially

Random labelling: label 1,000 randomly selected images. Most are easy cases the model already handles. You have paid for 1,000 labels and bought very little information. This is what most labelling budgets buy.

Active learning: label the 1,000 the model is least certain about, or gets wrong. Every label lands where the decision boundary is unclear.

Analogy — the apprentice. You would not train an apprentice by having them repeat tasks they already do correctly. You would put them on the jobs they find difficult, with someone watching. Active learning is that principle applied to a training set — and it works for the same reason.

Deeper detail — the link to Day 1. Active learning targets epistemic uncertainty, which is the kind more data can reduce. That is why it works and why it will not help with aleatoric uncertainty. If the model is uncertain because the task is genuinely ambiguous, labelling those cases adds contradictory examples rather than information. Separating the two, as on Day 1, tells you whether the loop will pay.

4.2 Sampling strategies

Strategy	Selects	Strength	Weakness
Uncertainty sampling	Predictions near the boundary	Simple, effective, cheap	Depends on calibrated confidence
Error-based	Cases where review overturned the model	Highest-information examples available	Needs a review loop running
Edge-case mining	Inputs unlike anything in training	Finds genuine coverage gaps	Can surface irrelevant outliers
Diversity sampling	Spread across feature space	Avoids 200 near-identical images	More complex to implement
Disagreement sampling	Ensemble members disagree	Directly targets epistemic uncertainty	Requires an ensemble
Class-targeted	Under-represented classes	Fixes known imbalance	Needs a target distribution

The dependency that is easily missed. Uncertainty sampling relies on calibrated confidence. If the model is over-confident — which, per Day 1, most neural networks are — it will select the wrong examples, and the whole loop underperforms for a reason nobody diagnoses. Calibrate before relying on it.

4.3 The production learning loop

Eight steps: predict, route, review, capture, select, retrain, validate, deploy.

The two steps that get skipped. **Capture** — teams build the review interface and never store the decision in a form that can reach training, so the most valuable data the system generates is discarded daily. And **validate** — teams retrain on newly collected data without an evaluation gate, so a batch of mislabelled reviews degrades the model with nobody noticing. The loop only compounds if both are present.

The question to ask each team. "Does your review decision get stored somewhere it can reach training?" The answer is usually no, and it is usually fixable in a week.

4.4 Feedback capture and label correction

1. **Capture the decision, not just the outcome.** What the reviewer decided, what the model proposed, the confidence, and where they disagreed.
2. **Make correcting easier than overriding.** If correcting takes four clicks and overriding takes one, you will get overrides and no training data. The interface determines the data quality.
3. **Attribute every correction.** Which reviewer, when, under which guideline version.
4. **Reconcile disagreements.** Route to a third reviewer or a defined adjudicator. Unreconciled disagreement is training noise.
5. **Sample-audit the corrections.** Corrections are labels and can be wrong.
6. **Close the loop visibly.** Tell reviewers when their corrections improved the model.

Why the last point decides whether the loop still runs in six months. "Correction rates decay within weeks if reviewers cannot see that it mattered. This is a human factor and it is decisive — more so than any technical choice in the loop."

4.5 Continuous evaluation and expert review

Six practices: keep a protected evaluation set; score it on a fixed schedule; refresh it from production carefully; route expert review deliberately to boundary and disputed cases; track reviewer agreement over time; and audit the corrections themselves.

The failure to avoid. Newly labelled production cases going into training *and* evaluation simultaneously. This quietly inflates every future measurement and it is Day 1's leakage lesson reappearing in operational form. Each new case goes to one or the other, never both.

Closing the Day

1. Diagnose before collecting — four of six common situations are not solved by more data.
2. Label consistency sets the ceiling.
3. Every augmentation is a claim about your process.
4. The evaluation gate is what makes MLOps real.
5. A feedback loop only compounds if capture and validation are both present.

Close by asking each team what their label agreement rate is. Most will not know, and that is the honest finding to carry into the capstone tomorrow.

Companion documents: **Day 4 Presentation** (39 slides with speaker notes) and **Day 4 Question Bank** (anticipated participant questions with model answers).